Drugi praktični test iz Algoritama i struktura podataka

Prvi praktični test održan 11.02.2026.

Ime i prezime: _____

Broj indeksa: _____

Instrukcije za rešavanje testa

1. Vreme izrade testa je **90 minuta**.
2. Ovo je *closed book test*, nije dozvoljeno korišćenje dodatnih materijala.
3. Pre početka testa:
    a. Vodite računa da imate sve spremno što vam je neophodno.
    b. Isključite mobilne telefone i odložite ih na katedru.
4. Ukupan broj bodova na testu je 25 bodova.
5. Na lokalnom disku se nalazi šablon koji je potrebno preuzeti i koristiti prilikom rešavanja zadataka. Detaljne instrukcije će biti date od strane dežurnog asistenta.
6. U pruženom šablonu obavezno upisati broj indeksa na naznačenom mestu.
7. LINQPad fajl koji se koristi preimenovati prema sledećem šablonu: **AISP2025_Test2_GrupaC_IT1g2024**
8. Ukoliko algoritam (rešenje zadatka) ima neku sintaksnu ili logičku grešku što uslovljava nemogućnost pokretanja i testiranja programa, najveći broj bodova koji se može osvojiti za taj algoritam je **50% od ukupnog broja bodova.**

Pre početka testa napišite ime, prezime i broj indeksa.

PREPISIVANJE TOKOM TESTA NIJE MORALNO I ETIČKI PRIHVATLJIVO!

Studenti koji budu prepisivali mogu biti udaljeni sa testa. Za studente koji budu prepisivali test će se smatrati nevažećim!

Ovim putem se izjašnjavam da sam upoznat/a sa pravilima ponašanja na testu kao i sa posledicama u slučaju kršenja istih.

Potpis studenta:_____

Zadatak 1 – *Završavanje implementacije BFS algoritma*

Osnovna metoda BFS algoritma nije potpuna. Dopuniti implementaciju ove metode koja treba da omogući kompletno izvršavanje ovog algoritma.

- Napisati algoritam.
- U *Main* metodi kreirati proizvoljan graf. Obezbediti način za testiranje i proveru nove funkcionalnosti, odnosno poziv ***BFS*** metode.

Dodatne napomene za rešavanje zadatka:

- U template-u je zakomentarisana metoda ***public void BFS(int startVertex).***

Zadatak 2 – *Pronalaženje karaktera sa maksimalnim brojem pojavljivanja*

Napisati metodu koja identifikuje karakter koji se najviše puta pojavljuje u stringu.

Dodatni zahtevi za rešavanje zadatka:

- Koristiti Hash tabelu iz šablona za čuvanje broja pojavljivanja svakog karaktera.
- Implementaciju pisati u metodi: ***public static void maxOccurrence(string str).***
- Razlika između malih i velikih slova **postoji**.
- Ukoliko postoji više karaktera koji se pojavljuju maksimalan broj puta, ispisati ih sve.

Primer:

Input 1:	*str = „Algoritmi i strukture"*
Output 1:	*„i" (pojavljuje se 3 puta)*
Input 2:	*str = „Vetar"*
Output 2:	*„Vetar" (svi karakteri se pojavljuju samo jednom)*

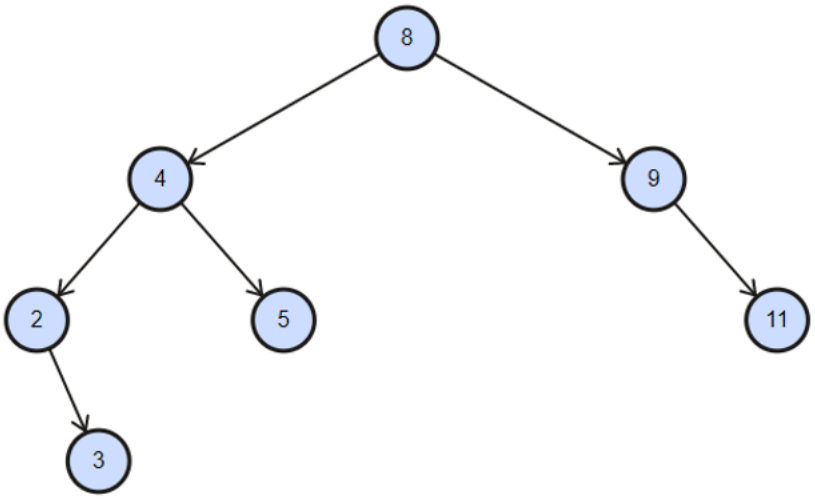
Zadatak 3 – *Putanja sa naizmeničnom parnošću (Zig-Zag Parnost)*

Napisati metodu koja proverava da li postoji putanja od korena do lista na kojoj se parni i neparni brojevi **naizmenično smenjuju** (npr. parni -> neparni -> parni...).

Dodatni zahtevi za rešavanje zadatka:

1. Javna metoda: ***public bool hasAlternatingPath().***
2. Privatna metoda: ***private bool hasAlternatingPath(Node current, bool expectedEven).***
3. Podrazumeva se da stablo ima barem dva čvora.
4. Algoritam **OBAVEZNO** pisati u okviru klase ***BinarySearchTree***

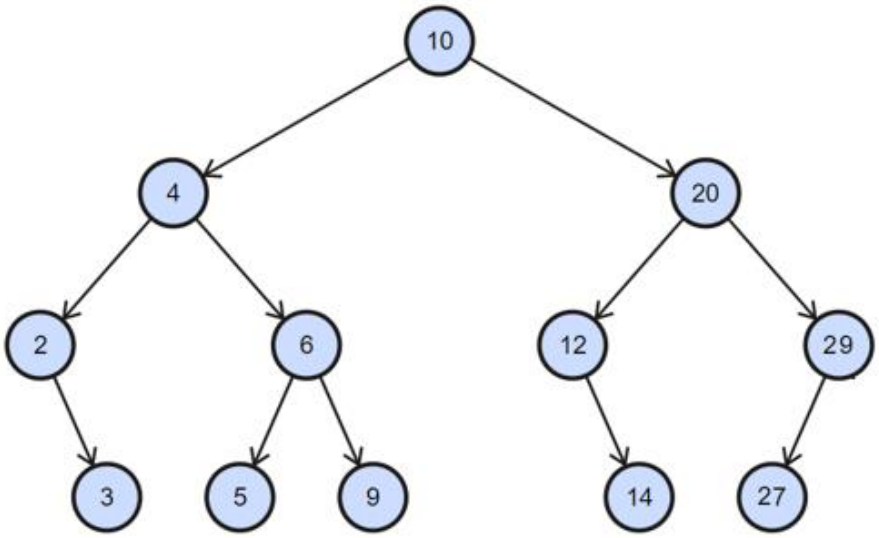
Primeri:



Slika 1 - Primer BST-a

Input 1: /

Output 1: *false*



Slika 2 - Primer BST-a

Input 2: /

Output 2: *false*